AVANTIKA UNIVERSITY

Library Management System

Student Names - Nikhil Kumawat
Vaidika Shivhare
Roll No: PU02425EUG11011
PU02425EUG11035

Course: Problem Solving using C++

Date: 4TH MAY 2026

Introduction

What is LMS?

A Library Management System is a software application that helps manage and organize library operations like adding books, issuing them, and tracking returns — all digitally.

Why is it Needed?

Libraries handle hundreds of books daily. Without a system, tracking becomes chaotic. LMS automates records, saves time, and reduces human error significantly.



Problem Statement

⏱️ **Time-Consuming**	Finding, recording and tracking books manually takes excessive time.
❌ **Error-Prone**	Human errors in data entry lead to misplaced or lost records.
📋 **Hard to Track**	No easy way to know which books are available or who has them.
📉 Inefficient Reports	Generating usage reports manually is nearly impossible.

Objectives

1.	Automate library operations like adding, issuing and returning books
2.	Reduce manual work and eliminate human errors
3.	Implement core OOP concepts: class, objects, encapsulation
4.	Provide a simple menu-driven interface for easy use

System Features

📚 Add Book

Add a new book with ID, title and author

🔍 Search Book

Search by title or author name instantly

📤 Issue Book

Issue a book to a student with their name

🗑️ Delete Book

Remove a book (blocked if currently issued)

📥 Return Book

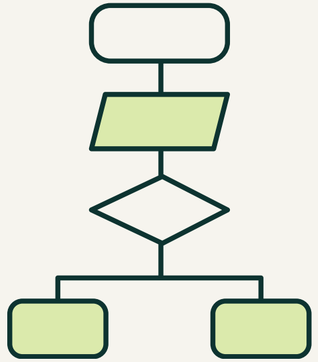
Mark a book as returned and available.

📋 View Status

See all available or currently issued books

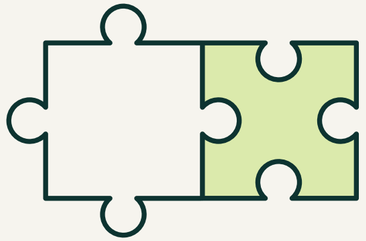


OOP Concepts Used



Encapsulation

Book data (id, title, author, status) is bundled inside the class.



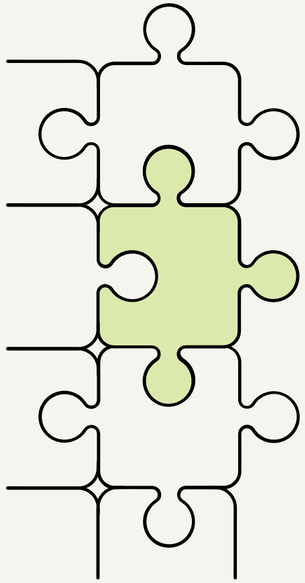
Abstraction

Users interact via a menu without needing to know internal logic.



Class & Object

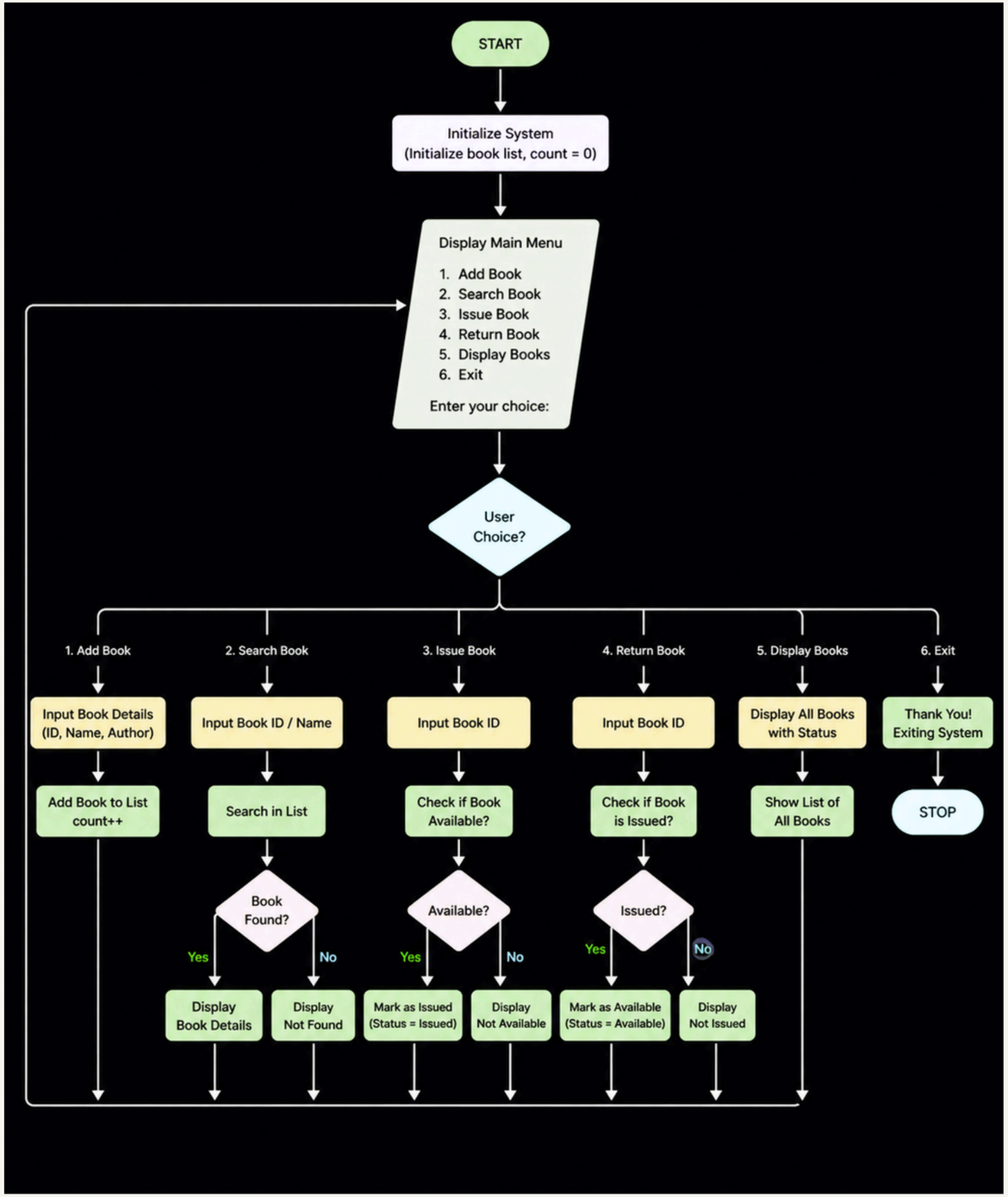
Book is defined as a class. Each book added is an object with its own data.



Modularity

Each operation (add, search, issue, return) is a separate function.

Flowchart :-


START
Initialize System
(Initialize book list, count = 0)
Display Main Menu
1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Exit
Enter your choice:
User Choice?
1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Exit
Input Book Details (ID, Name, Author)
Input Book ID / Name
Input Book ID
Input Book ID
Display All Books with Status
Thank You! Exiting System
Add Book to List count++
Search in List
Check if Book Available?
Check if Book is Issued?
Show List of All Books
STOP
Book Found?
Available?
Issued?
Yes
No
Yes
No
Yes
No
Display Book Details
Display Not Found
Mark as Issued (Status = Issued)
Display Not Available
Mark as Available (Status = Available)
Display Not Issued

Code Snippet

```cpp
#include <iostream>
#include <string>
using namespace std;

// 🔹 Class to represent a Book
class Book {
public:
    int id;             // Book ID
    string name;        // Book name
    string author;      // Author name
    bool issued = false; // Book status (false = available)

    // 🔹 Function to add book details
    void add() {
        cout << "Enter Book ID: ";
        cin >> id;
        cin.ignore(); // Clear input buffer

        cout << "Enter Book Name: ";
        getline(cin, name);

        cout << "Enter Author: ";
        getline(cin, author);

        issued = false; // New book is available
        cout << "Book added successfully!\n";
    }

    // 🔹 Function to display book details
    void show() {
        cout << "\nID: " << id
             << "\nName: " << name
             << "\nAuthor: " << author
             << "\nStatus: " << (issued ? "Issued" : "Available") << endl;
    }
};

int main() {
    Book b[100];  // Array to store up to 100 books
    int n = 0;    // Number of books currently stored
    int choice, id;
    bool found;   // Flag to check if book exists

    do {
        // 🔹 Display menu options
        cout << "\n\n--- Library Menu ---\n";
        cout << "1. Add Book\n2. Display Books\n3. Search Book\n4. Issue Book\n5. Return Book\n6. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch(choice) {

            // 🔹 Add a new book
            case 1:
                b[n].add(); // Call add function
                n++;        // Increase book count
                break;

            // 🔹 Display all books
            case 2:
                if(n == 0)
                    cout << "No books available.\n";
                else {
                    for(int i = 0; i < n; i++)
                        b[i].show(); // Display each book
                }
                break;

            // 🔹 Search for a book by ID
            case 3:
                cout << "Enter Book ID: ";
                cin >> id;
                found = false;

                for(int i = 0; i < n; i++) {
                    if(b[i].id == id) {
                        b[i].show(); // Show book details
                        found = true;
                        break;
                    }
                }

                if(!found)
                    cout << "Book not found.\n";
                break;

            // 🔹 Issue a book
            case 4:
                cout << "Enter Book ID to issue: ";
```

Code Snippet

```
87          // 🔹 Issue a book
88          case 4:
89              cout << "Enter Book ID to issue: ";
90              cin >> id;
91              found = false;
92
93              for(int i = 0; i < n; i++) {
94                  if(b[i].id == id) {
95                      if(!b[i].issued) {
96                          b[i].issued = true; // Mark as issued
97                          cout << "Book issued successfully.\n";
98                      } else {
99                          cout << "Book already issued.\n";
100                     }
101                     found = true;
102                     break;
103                 }
104             }
105
106             if(!found)
107                 cout << "Book not found.\n";
108             break;
109
110         // 🔹 Return a book
111         case 5:
112             cout << "Enter Book ID to return: ";
113             cin >> id;
114             found = false;
115
116             for(int i = 0; i < n; i++) {
117                 if(b[i].id == id) {
118                     if(b[i].issued) {
119                         b[i].issued = false; // Mark as available
120                         cout << "Book returned successfully.\n";
121                     } else {
122                         cout << "Book was not issued.\n";
123                     }
124                     found = true;
125                     break;
126                 }
127             }
128
129             if(!found)
130                 cout << "Book not found.\n";
131             break;
```

```
132
133             // 🔹 Exit program
134             case 6:
135                 cout << "Exiting...\n";
136                 break;
137
138             // 🔹 Invalid input handling
139             default:
140                 cout << "Invalid choice!\n";
141         }
142
143     } while(choice != 6); // Loop until user exits
144
145     return 0; // End of program
146 }
```

Output Screens

Main Menu

```
--- Library Menu ---
1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit
Enter choice:
```

Add & Issue Book

```
--- Library Menu ---
1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit
Enter choice: 1
Enter Book ID: 456789
Enter Book Name: too good to be true
Enter Author: prajkata koli
Book added successfully!
```

- Search book

```
--- Library Menu ---
1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit
Enter choice: 3
Enter Book ID: 456789

ID: 456789
Name: too good to be true
Author: prajkata koli
Status: Available
```

Limitations

No Database	Data is stored in arrays and lost when the program closes. No file or database persistence.
No GUI	The interface is text-based (console). No buttons, windows or graphical elements.
Small Scale	Fixed array of 100 books. Not suitable for large libraries with thousands of records.
No Login System	Any user can access all features. No admin/student role separation or authentication.

Future Scope

🗄️ **File / Database Storage**	Save all book data to files or connect to a database like MySQL so records persist after the program closes.
🖥️ **Graphical Interface (GUI)**	Replace the console menu with a GUI using Qt or a web-based frontend for a better user experience.
🔐 **Login & Role System**	Add admin and student login with separate permissions and access control.
📅 **Due Date & Fine System**	Track issue dates and calculate overdue fines automatically for late returns.

Conclusion

- Successfully automated core library operations using C++
- Demonstrated practical use of OOP: Classes, Objects, Encapsulation
- Menu-driven interface makes the system accessible to any user
- Lays a strong foundation for advanced features like databases and GUI

Thank You!

